

UE-INF2321M Animation, Corps Articulés Et Moteurs Physiques



Université Claude Bernard



Lyon 1

COTTIER Alexandre, P2018147 → vidéo démo :

<https://drive.google.com/drive/folders/1cVRgldJz7exwME5FBPBB2q-q8ZztaGb4?usp=sharing>

Y a une partie dont je suis repartie de zero et une partie dont je me suis servie de code que j'avais déjà fait en M1 pour l'option d'animation, je n'ai évidemment pas fait le même sujet que l'année dernière.

J'ai fait 3 sujets (Rigid body, SPH et un dernier sujet inspiré d'une vidéo)

Partie Simulation d'un objet rigide:

Initialisation du repère local des sommets : calcul de $_ROI$ dans `initObjetSimule()` :

Je construis un tableau $_ROI$ qui stocke, pour chaque particule/sommet i , sa position dans le repère de l'objet, relativement au centre de masse :

$$r_{0i} = p_i - x$$

- p_i : position initiale du sommet i (lue depuis le fichier)
- x : position du rigid body
- je dimensionne $_ROI$ à la taille du nombre de particules,
- je remplis $_ROI[i] = \text{particule} \rightarrow \text{GetPosition}() - _Position$.

Un objet rigide impose que chaque sommet suive la transformation rigide :

$$p_i(t) = R(t) r_{0i} + x(t)$$

- $R(t)$: rotation de l'objet
- $x(t)$: translation
- r_{0i} : coordonnée locale du sommet

Construction du maillage affichable : initMeshObjet() :

- Création du mesh :
 - `m_ObjetSimule = Mesh(GL_TRIANGLES);`
- Couleur par défaut :
 - `m_ObjetSimule.color(Color(1, 1, 200));`
- Remplissage du mesh :
 - parcours les indices de rendu $i = 0.._VSize-1$
 - récupères l'index réel `index = _VIndices[i]`
 - ajoutes :
 - la normale `_vectNormals[index]`
 - le sommet `P[index]` (position du sommet)

Mise à jour du mesh pendant l'animation : updateVertex() :

Je mets à jour chaque sommet i :

- `index = _VIndices[i]`
- normal mise à jour :
 - `m_ObjetSimule.normal(i, _vectNormals[index])`
- position mise à jour :
 - `m_ObjetSimule.vertex(i, P[index].x, P[index].y, P[index].z)`
- `P[index]` contient la position "physique" courante du sommet `index` (après la transformation rigide).
- Comme le mesh contient `_VSize` sommets "rendus", je met à jour les sommets du mesh un par un, en réutilisant l'indexation `_VIndices[]`.

Activation de la gestion des collisions dans Simulation()

Dans ta boucle de simulation d'un pas :

1. je mets à jour l'état dynamique avec `Solve()`
2. j'applique ensuite une correction de collision (projection / correction de vitesse) via `Collision()`

Calcul de la masse totale et du barycentre — CalculMasse() :

J'ai fait le calcul de la masse totale M de l'objet rigide et de son barycentre G .

- Je parcours toutes les particules du maillage.
- Chaque particule a une masse m_i et une position p_i .
- La masse totale est :

$$M = \sum_i m_i$$

- Le barycentre est :

$$G = \frac{1}{M} \sum_i m_i p_i$$

Si $M = 0$, je retombe sur une valeur par défaut (ici la position courante de l'objet).

Calcul de l'état dynamique — CalculStateX() :

Je fais la mise à jour des grandeurs dérivées de l'état :

- La vitesse linéaire est obtenue depuis la quantité de mouvement :

$$v(t) = \frac{P(t)}{M}$$

- La vitesse angulaire ω est obtenue depuis le moment cinétique $L(t)$ via l'inverse du tenseur d'inertie exprimé dans le repère monde :

$$I^{-1}(t) = R(t) I_{\text{body}}^{-1} R(t)^T$$
$$\omega(t) = I^{-1}(t) L(t)$$

Dans le code je fais :

- calcul de $I^{-1}(t)$ par $R I_{\text{body}}^{-1} R^T$
- puis $\omega = I^{-1} L$
- et $v = P/M$

Calcul des dérivées de l'état — CalculDeriveeStateX(g)

Calcul des forces/torques et de la dérivée de la rotation.

- Force extérieure : uniquement la gravité

$$F(t) = M g$$

- Couple extérieur : mis à zéro

$$\tau(t) = 0$$

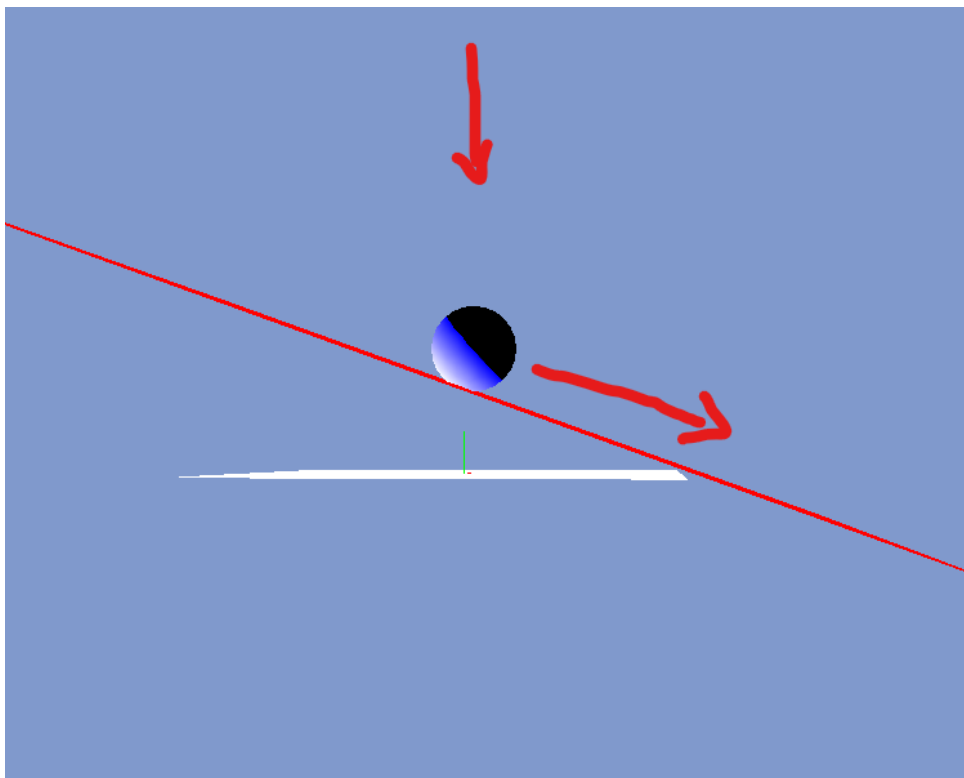
- Dérivée de la rotation :

j'utilise la matrice antisymétrique $W(\omega)$ construite à partir de $\omega = (\omega_x, \omega_y, \omega_z)$:

$$W(\omega) = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}$$

et :

$$\dot{R}(t) = W(\omega) R(t)$$



Intégration temporelle (Euler semi-implicite) — Solve(visco) :

J'ai complété le schéma d'intégration.

Mise à jour impulsionnelle (momentum) :

$$\begin{aligned}P(t + \Delta t) &= P(t) + \Delta t F(t) \\L(t + \Delta t) &= L(t) + \Delta t \tau(t)\end{aligned}$$

Amortissement viscosité :

J'applique un amortissement simple :

$$P \leftarrow (1 - \text{visco})P, \quad L \leftarrow (1 - \text{visco})L$$

Mise à jour :

$$\begin{aligned}v(t + \Delta t) &= \frac{P(t + \Delta t)}{M} \\x(t + \Delta t) &= x(t) + \Delta t v(t + \Delta t) \\R(t + \Delta t) &= R(t) + \Delta t \dot{R}(t)\end{aligned}$$

Stabilisation de la rotation

Après intégration, la matrice R dérive numériquement et n'est plus orthonormale. J'ai ajouté une ré-orthonormalisation :

- normalisation du premier axe c_0
- orthogonalisation puis normalisation de c_1
- reconstruction de $c_2 = c_0 \times c_1$

Ça garantit que R reste une rotation valide (évite les "étirements" du rigide).

Mise à jour des sommets affichés

Enfin, pour chaque sommet i , je recalcule sa position monde :

$$r_i(t + \Delta t) = R(t + \Delta t) r_{0i} + x(t + \Delta t)$$

et je met à jour le tableau $P[i]$.

Collision avec le sol — Collision() :

J'ai ajouté une gestion de collision entre l'objet et un plan incliné/droit (j'ai fait plan normal sol au début puis j'ai fait avec un plan incliné pour voir l'objet glissé).

Géométrie du plan :

Le plan est défini par :

- un point p_0
- une normale unitaire n

La distance signée du centre de masse x au plan est :

$$d = n \cdot (x - p_0)$$

Correction de pénétration

Si $d < 0$, l'objet est "sous" le plan $\Rightarrow$ je le remonte :

$$x \leftarrow x - d n$$

Réponse en vitesse (pas de rebond + frottement)

On décompose la vitesse :

$$v_n = (v \cdot n) n, \quad v_t = v - v_n$$

Si l'objet rentre dans le plan ($v \cdot n < 0$) :

- restitution $e = 0 \Rightarrow$ pas de rebond (j'ai un objet très dur):

$$v_n^{new} = -e v_n = 0$$

- frottement simple sur la tangente :

$$v_t \leftarrow (1 - \mu) v_t$$

Puis :

$$v \leftarrow v_t + v_n^{new}$$

Synchronisation de l'état impulsif et du mesh

Après modification de v , je mets à jour :

$$P = Mv$$

et je recalcule les sommets :

$$r_i = R r_{0i} + x$$

CalculAccel_ForceGravite — calcul des accélérations

Je fais:

$$\frac{f_i}{\rho_i}$$

L'accélération devient directement :

$$a_i = \frac{f_i}{\rho_i} + g$$

Implémentation :

- Pour chaque particule i :
 - $A[i] = \text{Force}[i] + g$;

Solve — intégration temporelle (Euler semi-implicite + viscosité) :

Le schéma d'intégration de l'état (vitesse et position) au pas de temps suivant.

J'utilise un Euler semi-implicite, qui fait :

1. Mise à jour des vitesses avec l'accélération :

$$v_i(t + \Delta t) = v_i(t) + \Delta t a_i(t)$$

2. Amortissement / viscosité :

$$v_i(t + \Delta t) \leftarrow (1 - \nu) v_i(t + \Delta t)$$

où ν = visco

3. Mise à jour des positions avec la vitesse mise à jour :

$$x_i(t + \Delta t) = x_i(t) + \Delta t v_i(t + \Delta t)$$

Implémentation:

- Boucle sur chaque sommet/particule :
 - $V[i] = V[i] + dt * A[i];$
 - si visco > 0 alors $V[i] *= (1 - visco);$
 - $P[i] = P[i] + dt * V[i];$

Partie Simulation d'un fluide avec la méthode SPH :

Calcul des densités : CalculDensite()

La densité au point i est estimée en lissant les masses des particules autour, via un noyau W de rayon h .

$$\rho_i = \sum_j m_j W(r_{ij}, h) \text{ avec } W(r, h) = \frac{4}{\pi h^8} (h^2 - r^2)^3 \text{ si } r < h$$

avec $r_{ij} = \| p_i - p_j \|$.

Pré-calcule la constante :

$$C = \frac{4}{\pi h^8}$$

- Pour chaque particule i , je boucle sur toutes les particules j .
- Si $r^2 < h^2$, j'ajoute :

$$\rho_i += m_j \cdot C \cdot (h^2 - r^2)^3$$

- Stocke le résultat dans rho[i].

Forces d'interaction : CalculInteraction(float visco)

Après avoir les densités, je calcule les forces dues au fluide :

- pression
- viscosité

Mise à jour densités + reset des forces

- CalculDensite();
- Force[i]=0

Boucle sur voisins (ici “voisins” = toutes particules, filtrées par $r < h$)

- $r = \| p_i - p_j \|$
- $q = r/h$
- Interaction seulement si $0 < r < h$

Terme de pression

$$15 \cdot bulk \cdot (\rho_i + \rho_j - 2\rho_0) \cdot \frac{1-q}{q} \cdot (p_i - p_j)$$

- bulk $\rightarrow$ “rigidité de compressibilité” (plus grand $\rightarrow$ fluide plus incompressible, donc plus raide)

Terme de viscosité

J’ajoutes un terme de viscosité basé sur la vitesse relative :

$$-40 \cdot visco \cdot (v_i - v_j)$$

Assemblage final + normalisations

Je multiplie pression+viscosité par un coefficient :

$$scalarPart = \left(\frac{m_j}{\rho_j} \right) \cdot \frac{1}{\pi h^4} \cdot (1 - q)$$

puis j’accumule sur i .

Et je stocke:

$$Force[i] = \frac{f_{int}}{\rho_i}$$

Force[i] devient l’accélération due aux interactions (sans la gravité).

Collisions / frontières : damp_reflect() + Collision()

Je veux empêcher les particules de sortir du domaine, en simulant une réflexion amortie (coefficient de restitution < 1).

damp_reflect(frontiere, barrier, indice_part)

- damping = 0.75
- Selon la frontière (0→x, 1→y, 2→z) :
 - Si la particule dépasse barrier, je repositionne la coordonnée sur la barrière (x = barrier)
 - J'inverse la vitesse correspondante en l'amortissant :

$$v = -damping \cdot v$$

Ça fait un rebond, ça rebondit mais perd de l'énergie.

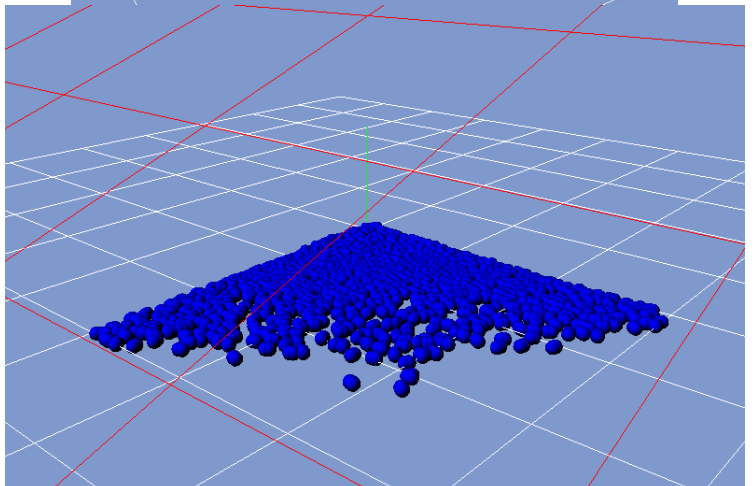
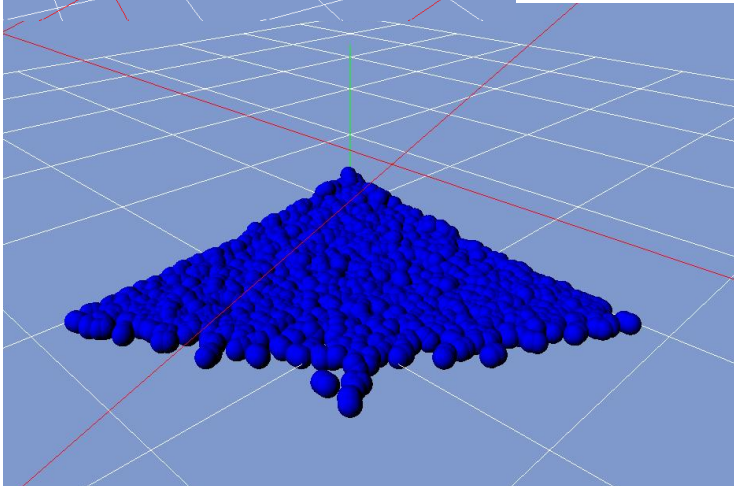
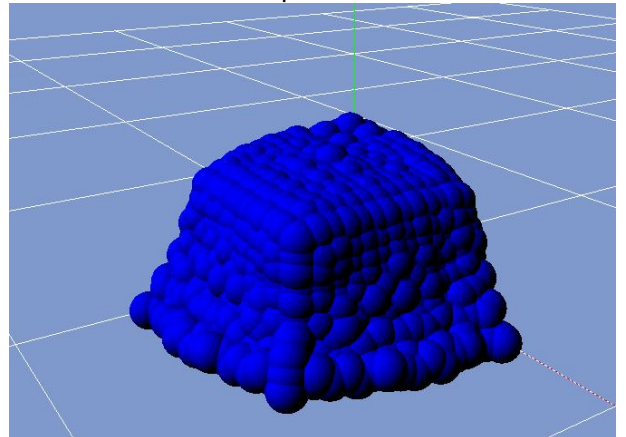
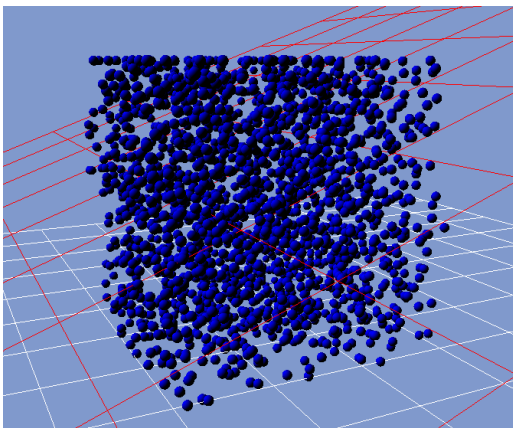
Collision()

J'applique cette logique à toutes les particules, sur une boîte :

$$x, y, z \in [0,3]$$

- si $P[i].x < 0 \rightarrow$ mur gauche
- si $P[i].x > 3 \rightarrow$ mur droit
- pareil pour y et z

Dans les images si dessous, la premiere est un echec a cause des paramètres



Partie π collisions (pas dans les sujets mais inspiré d'une vidéo voir a la fin) :

L'objectif était d'implémenter un mode de simulation événementielle 1D (deux blocs/spheres + un mur, inspiré d'une vidéo de 3blue1brown) permettant de compter le nombre de collisions élastiques entre deux objets rigides et un mur. Ce nombre de collisions est lié à l'approximation de π (expérience : ratio de masses $100^n \rightarrow$ collisions $\approx$ premières décimales de π).

Concrètement :

- Deux objets rigid body se déplacent sur l'axe x uniquement (je simplifie parce que j'ai voulu faire ce sujet juste pour implémenter cet algo pour le fun plus ou moins).
- Un mur vertical est placé en $x = x_{\text{mur}}$.
- On compte chaque collision :
 - bloc $\leftrightarrow$ mur
 - bloc $\leftrightarrow$ bloc
- On affiche ensuite le nombre total de collisions (ex : 31 pour un ratio de masses 100).

Ajout du paramètre "*massscale*" sur l'objet rigide :

J'ai ajouté un nouveau paramètre dans le fichier de paramètres d'un rigid body :

- `massscale = ...`

Il permet de multiplier la masse totale de l'objet rigide sans modifier le maillage ni les masses unitaires de chaque particule du MSS interne.

Dans `CalculMasse()` je calcule :

$$M = \sum_i m_i$$

puis j'applique :

$$M \leftarrow M \cdot \text{massscale}$$

J'ai ajouté des getters/setters simplifiés :

- `mass()` : masse totale M
- `x()` : position du centre de masse x
- `vx()` : vitesse sur x calculée à partir de la quantité de mouvement
- `setX(nx)` : fixe la position x
- `setVx(nvx)` : fixe v_x et met à jour la quantité de mouvement $P_x = Mv_x$
- `setRotationIdentity()` : force une rotation identité (pas de rotation) pour stabiliser le mode 1D
- `syncFromState()` : reconstruit les positions des sommets (tableau P) à partir de `_Positionet_Rotation`

Relation vitesse / quantité de mouvement

Le code suit la relation :

$$\vec{v} = \frac{\vec{P}}{M}$$

donc :

$$v_x = \frac{P_x}{M}$$

Si la masse est très grande, un même *quantitemouv* donne une vitesse très petite.

Simulation “ π collisions” dans `Simulation()`

La simulation de base avance par pas de temps Δt , ce qui peut “rater” des collisions si Δt est trop grand.

Ici je veux un comportement exact en 1D, donc je calcule directement le temps jusqu’au prochain événement (collision).

Variables physiques utilisées :

Je modélise deux blocs (enfin sphère dans la démo) A et B , chacun défini par :

- masse m
- position x
- vitesse v

On conserve toujours A à gauche et B à droite (tri par position).

Collision avec le mur (mur à gauche) :

Le mur est en $x = x_{\text{mur}}$.

Si A va vers la gauche ($v_A < 0$), le temps d'impact est :

$$t_{\text{mur}} = \frac{x_A - x_{\text{mur}}}{-v_A}$$

Collision entre les deux blocs :

Si A rattrape B (vitesse relative $v_A - v_B > 0$), alors :

$$t_{\text{bloc}} = \frac{x_B - x_A}{v_A - v_B}$$

On prend le prochain événement :

$$t_{\text{next}} = \min(t_{\text{mur}}, t_{\text{bloc}}, \text{temps restant})$$

Puis j'avance :

$$x_A \leftarrow x_A + v_A \cdot t_{\text{next}}, \quad x_B \leftarrow x_B + v_B \cdot t_{\text{next}}$$

Mise à jour des vitesses lors des collisions :

Collision élastique avec le mur

Un mur rigide renvoie :

$$v_A \leftarrow -v_A$$

On incrémente le compteur de collisions.

Collision élastique 1D bloc $\leftrightarrow$ bloc

Avec vitesses avant choc u_1, u_2 et masses m_1, m_2 , les vitesses après choc sont :

$$v_1 = \frac{m_1 - m_2}{m_1 + m_2} u_1 + \frac{2m_2}{m_1 + m_2} u_2$$
$$v_2 = \frac{2m_1}{m_1 + m_2} u_1 + \frac{m_2 - m_1}{m_1 + m_2} u_2$$

J'incrémente le compteur de collisions.

Condition d'arrêt :

La simulation s'arrête lorsque :

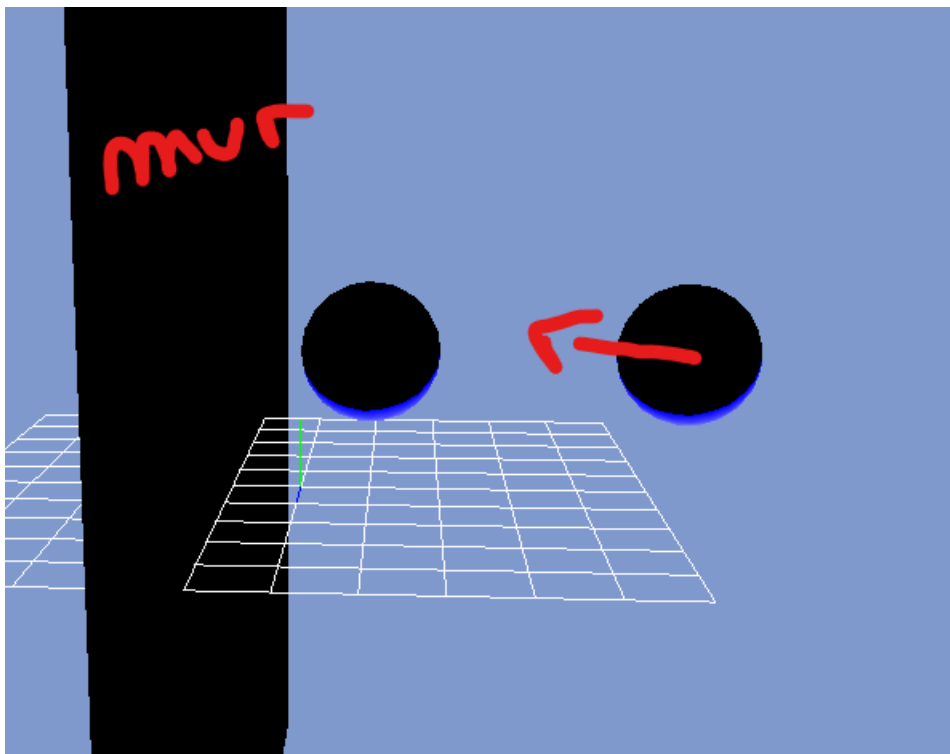
- les deux vitesses sont vers la droite ($v_A \geq 0$ et $v_B \geq 0$)
- et le bloc de droite ne rattrapera plus le bloc de gauche ($v_B \geq v_A$)

Ce qui signifie qu'il n'y aura plus jamais de collision ni avec le mur, ni entre les blocs.

Résultat obtenu :

Avec un ratio de masses 100: 1, j'obtiens :

- 31 collisions, ce qui correspond au comportement attendu (approximation de π : 3.1...).



Remarques :

- La collision du mode π est 1D (sur x uniquement) : elle ne prend pas en compte les triangles, ni une vraie collision 3D.
- La simulation repose sur les centres de masse (et éventuellement une demi-largeur si je l'ajoute).
Visuellement, il peut y avoir une pénétration si je ne corrige pas le "contact" par un rayon.
- Ce mode π est volontairement indépendant du reste de la simulation 3D (gravité, plan incliné, collisions de base), afin d'avoir un comptage stable.

Ressource :

Lien de la vidéo : <https://www.youtube.com/watch?v=HEfHFsfGXjs>

Amélioration ? :

Concernant les améliorations possibles il y en a plein, déjà pour la partie SPH je sais que ce n'est pas optimale du tout, j'ai du des rendu bien meilleur que le miens, j'aurais bien aimé implémenter d'autres choses encore en lien avec mon mode π mais le temps ne m'en laisse pas l'occasion malheureusement, c'est vrai que c'est un peu hors sujet peut être mais j'ai voulu le rajouter car j'avais envie de l'implémenter et que c'était l'occasion idéale.

J'aurais aussi pu faire de la parallélisation CPU/GPU $\rightarrow$ calculs par particule, recherche de voisins (SPH), ressorts (MSS), collisions (broadphase).